

SDN and Openflow

Programmable Networks A.Y. 2024/25

Outline

- Introduction
- Generalized Forwarding
- SDN Paradigm
- Openflow Protocol
- Data/Control Plane interactions in SDN

Outline

- Introduction
- Generalized Forwarding
- SDN Paradigm
- Openflow Protocol
- Data/Control Plane interactions in SDN

Introduction

- User requirements are changing fast, asking for always more performing and flexible services (bandwidth, latency, reliability, etc.)
- To meet these stringent requirements, innovative solutions are required
- ... unfortunately it is hard to change the way the Internet work
 - Internet ossification problem
 - Closed equipment
 - Over specified
 - Few people can innovate
- A paradigm shift on the way networks are managed and controlled is required

Introduction

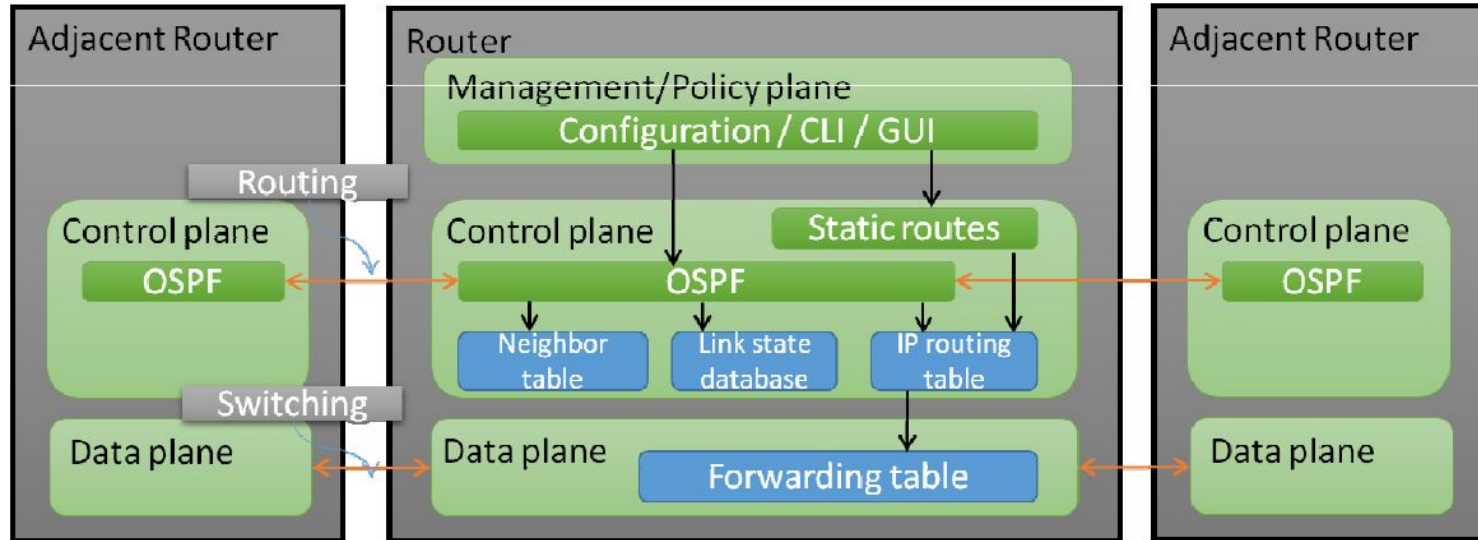
- The **Software Defined Networking** (SDN) paradigm aims at achieving the flexibility of the software in the provisioning of new network services
- The key idea is to **decouple the network services from the underlying hardware**
- This is achieved by
 - introducing the concept of **Generalized Forwarding**
 - **decoupling** Data and Control planes
 - defining a standard and **open interface** to instruct the network devices

Traditional Networking

- **Data Plane:** responsible of processing and forwarding of packets
 - Based on state in routers and endpoints
 - E.g., IP, TCP, Ethernet, etc.
 - Fast timescales (**per-packet**)
- **Control Plane:** decision – responsible of establishing the state in routers
 - Determines how and where packets are forwarded
 - Routing, traffic engineering, firewall state, ...
 - Slow time-scales (**per control event**)
- **Management plane:** configuration – responsible of general device behavior
 - Determines how the control plane should be configured
 - Slow time-scales (**manual configuration by network administrators**)

Traditional Networking

- In an IP router, the control plane role is played by dynamic routing protocols and the associated state
 - E.g. OSPF and the Link State Database



Traditional Networking

	Data	Control	Management
Time scales	Packets	Events	Humans
Task	Forwarding/ buffering/ filtering/ scheduling	Routing, circuit set-up	Analysis, configuration
Location	Hardware • Specialized hardware • Processes at line rate • Every packet • Very fast	Router software • Uses CPU • Can only process a small number of packets • Very slow	Human or perl scripts

Outline

- Introduction
- Generalized Forwarding
- SDN Paradigm
- Openflow Protocol
- Data/Control Plane interactions in SDN

Generalized Forwarding

- **flow:** defined by header fields
- **generalized forwarding:** simple packet-handling rules
 - **Pattern:** match values in packet header fields
 - **Actions:** for matched packet: drop, forward, modify, matched packet or send matched packet to controller
 - **Priority:** disambiguate overlapping patterns
 - **Counters:** #bytes and #packets



Flow table in a router (computed and distributed by controller) define router's match+action rules

Generalized Forwarding

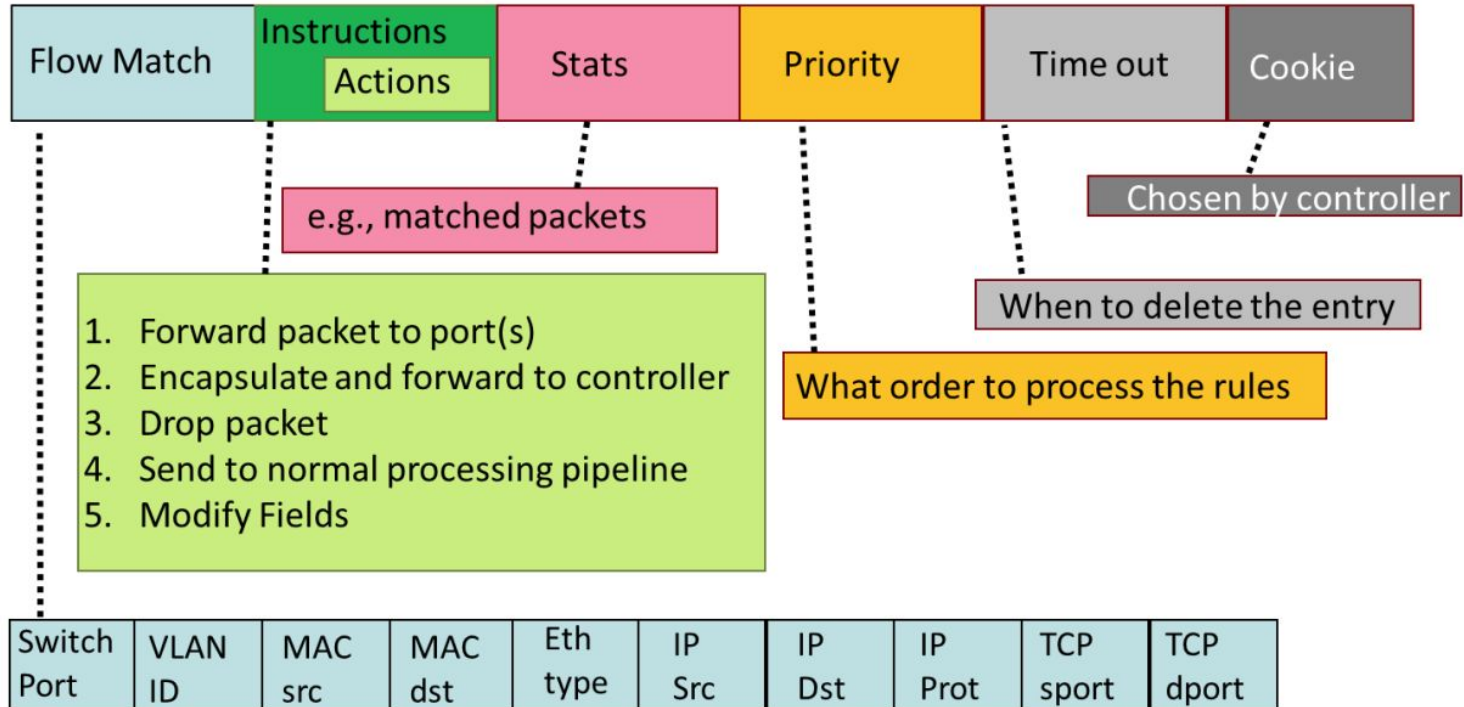
- **flow:** defined by header fields
- **generalized forwarding:** simple packet-handling rules
 - **Pattern:** match values in packet header fields
 - **Actions:** for matched packet: drop, forward, modify, matched packet or send matched packet to controller
 - **Priority:** disambiguate overlapping patterns
 - **Counters:** #bytes and #packets



* : wildcard

1. src=1.2.*.*, dest=3.4.5.* → drop
2. src = *.*.*.*, dest=3.4.*.* → forward(2)
3. src=10.1.2.3, dest=*.*.*.* → send to controller

Flow Table Entries



Switching

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Instructions
*	*	00:1f:...	*	*	*	*	*	*	*	Actions
										port6

Flow Switching

										Instructions
Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Actions
port3	00:20:..	00:1f:..	0800	vlan1	1.2.3.4	5.6.7.8	4	17264	80	port6

Firewall

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Instructions
*	*	*	*	*	*	*	*	*	22	drop
										Actions

Examples

Routing

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action	Instructions
*	*	*	*	*	*	5.6.7.8	*	*	*	port6	

VLAN Switching

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action	Instructions
*	*	00:1f..	*	vlan1	*	*	*	*	*	port6, port7, port9	

NAT

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action	Instructions
*	00:1f..	*	*	*	1.2.3.4	*	*	*	*	Set ip 3.4.5.6, port9	

Outline

- Introduction
- Generalized Forwarding
- **SDN Paradigm**
- Openflow Protocol
- Data/Control Plane interactions in SDN

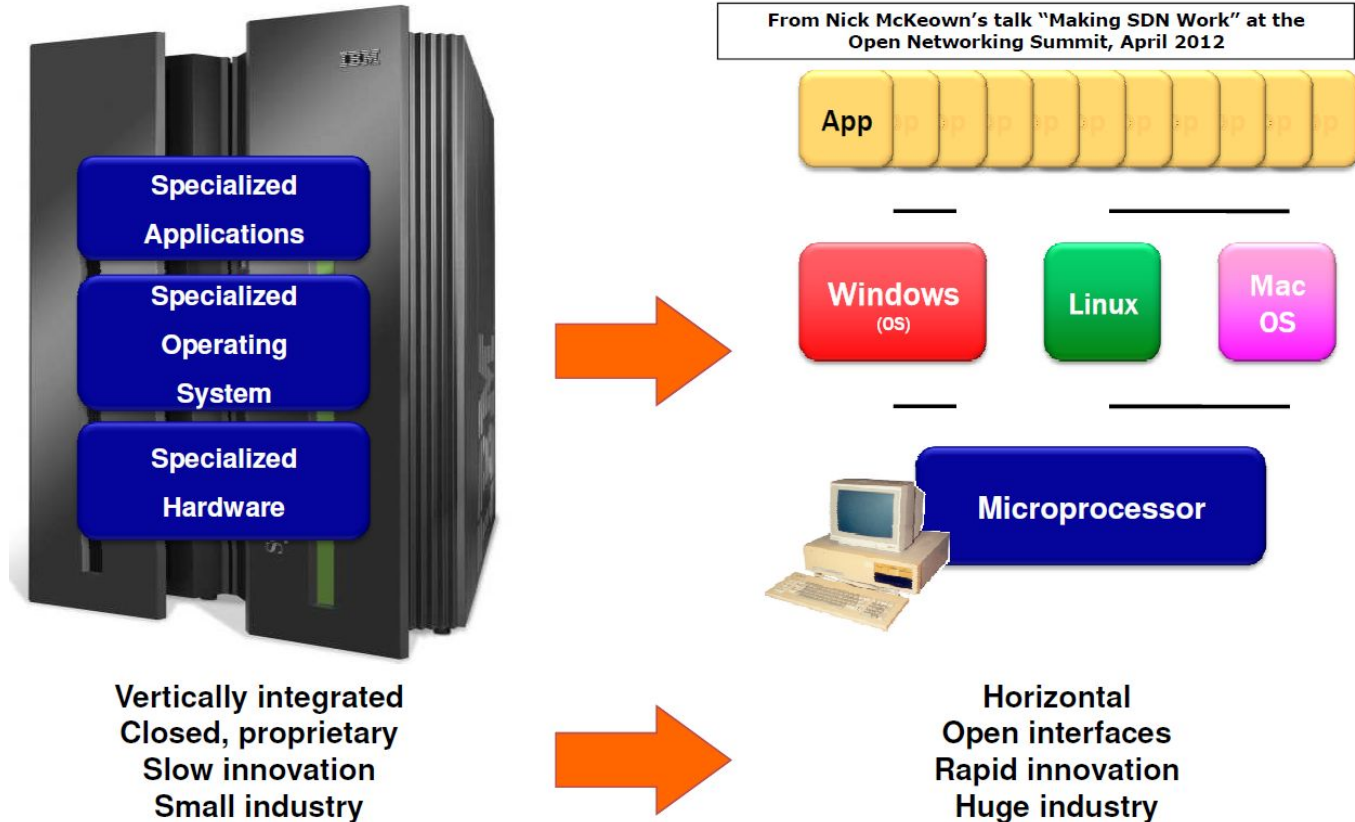
SDN switches

- All the different **network functions** may be **replaced by “generic” SDN switches** whose behaviour is programmed by the control plane
- By **decoupling the network function from the physical infrastructure**, the SDN approach is also useful to second another emerging trend in the telecommunications industry: **Network Function Virtualization (NFV)**

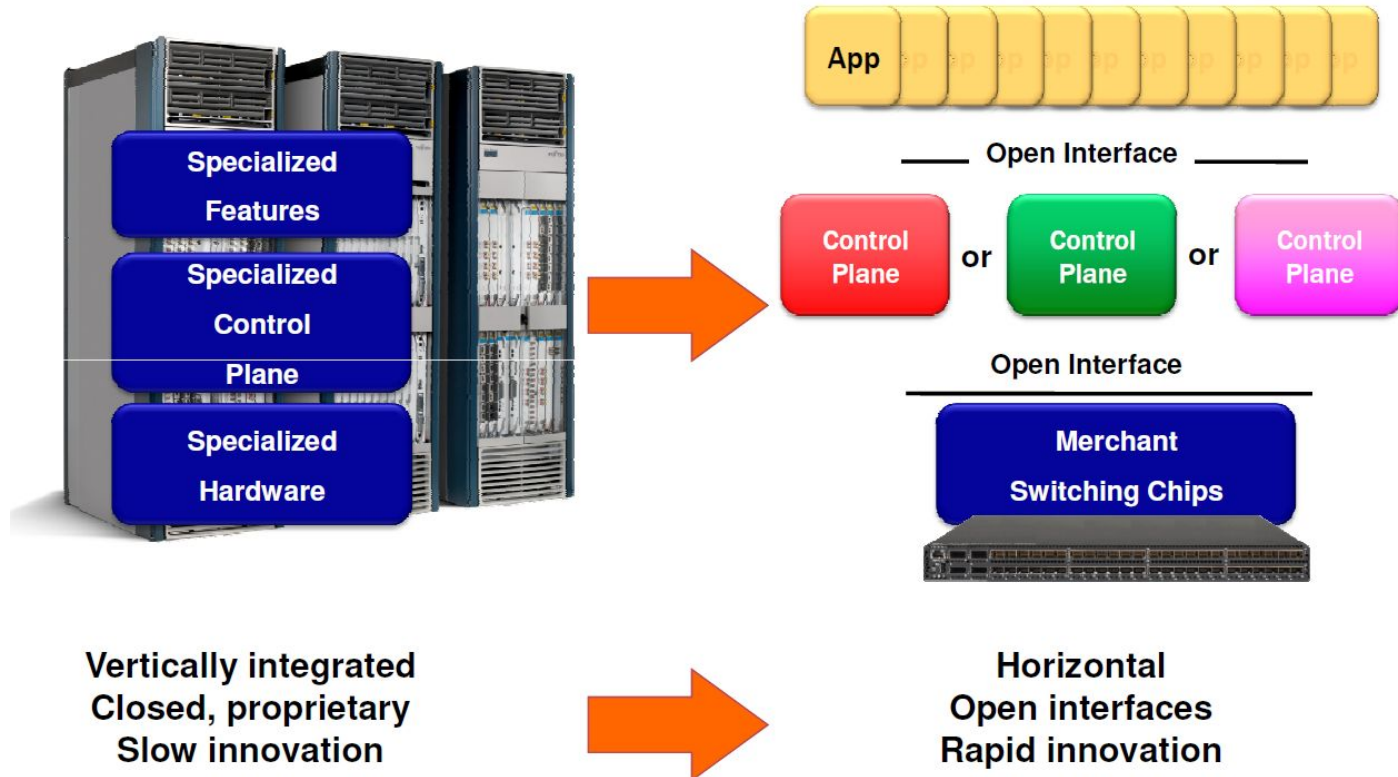
SDN switches

- SDN allows to unify different kinds of boxes
 - Router
 - **Match:** longest destination IP prefix
 - **Action:** decrement TTL, re-compute header checksum, forward out a link
 - Switch
 - **Match:** destination MAC address
 - **Action:** forward or flood
 - Firewall
 - **Match:** IP addresses and TCP/UDP, port numbers
 - **Action:** permit or deny
 - NAT
 - **Match:** IP address and port
 - **Action:** rewrite address and port

SDN: decoupling hardware and software

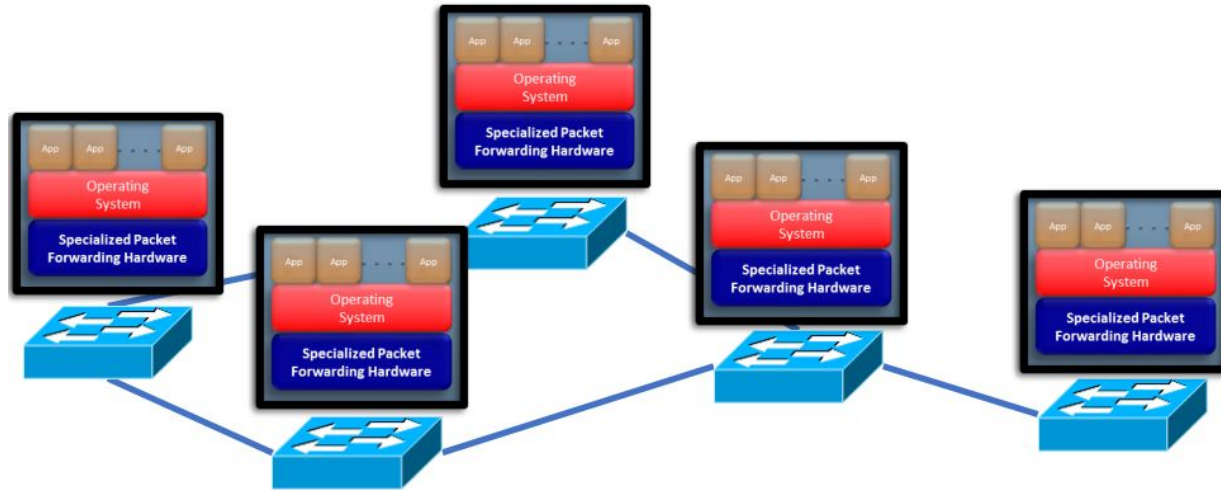


SDN: decoupling hardware and software



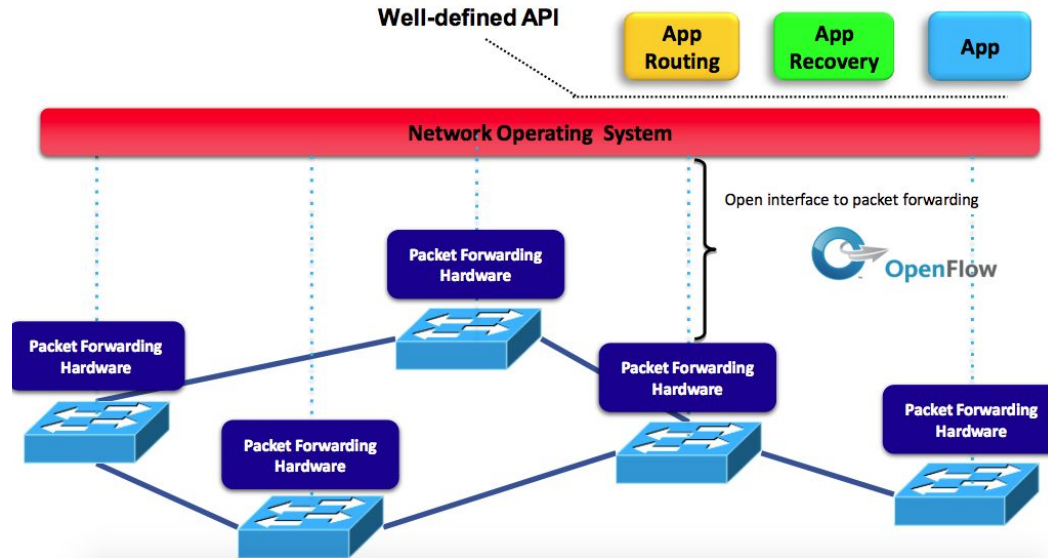
SDN: network OS

- In today's network devices, the software is tightly coupled with the hardware



SDN: network OS

- The whole network as one big machine
- The **key is to have a standardized control interface that speaks directly to hardware**

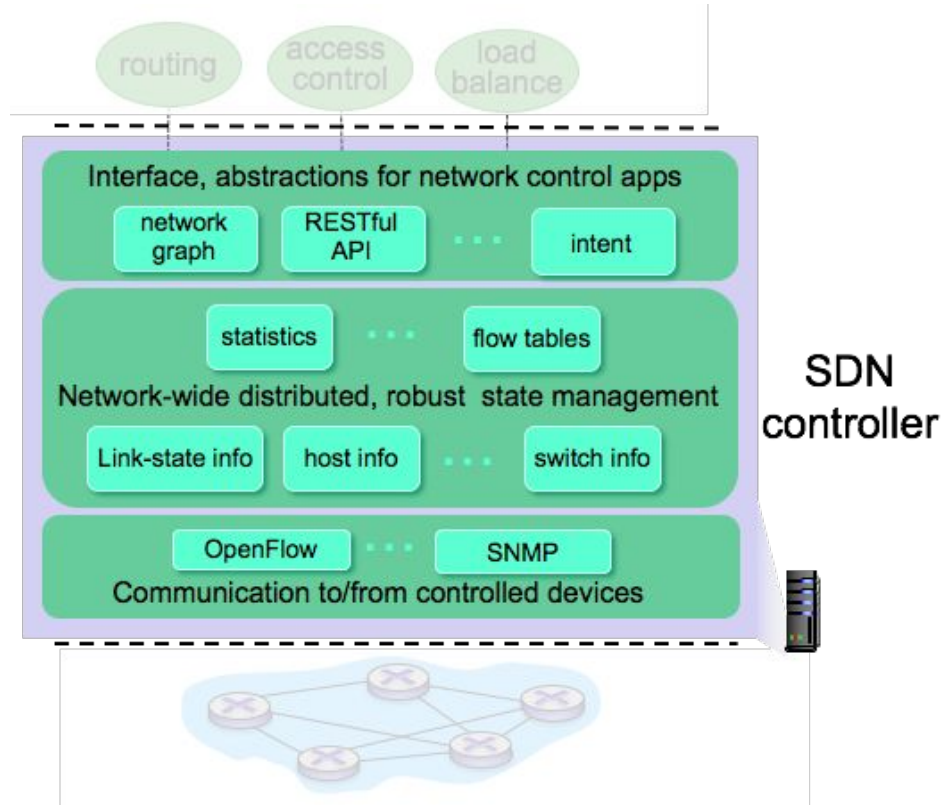


The SDN Controller

- **Separate control plane and data plane entities**
 - Network intelligence and state are logically centralized
 - The underlying network infrastructure is abstracted from the applications
- **Remotely control network devices from a central entity**
- **Execute or run control plane software on general purpose hardware**
 - Decouple from specific networking hardware
 - Use commodity servers
- **An architecture to control**
 - not just a networking device ...
 - but an entire network

Components of SDN controller

- **Interface layer to network control apps:** abstractions API
- **Network-wide state management layer:** state of networks links, switches, services: **a distributed database**
- **communication layer:** communicate between SDN controller and controlled switches

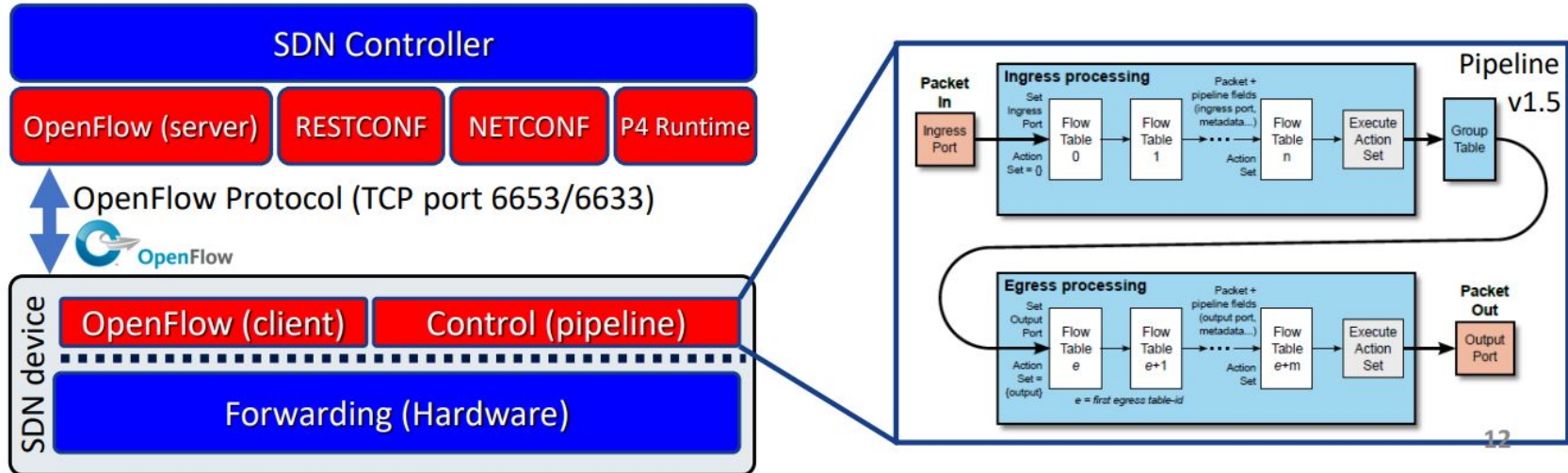


Outline

- Introduction
- Generalized Forwarding
- SDN Paradigm
- Openflow Protocol
- Data/Control Plane interactions in SDN

OpenFlow basics

- OpenFlow is the protocol (one of the protocols) used for **communication from the controller and the forwarding data plane**
- OpenFlow introduces a standard way to control switch/routers

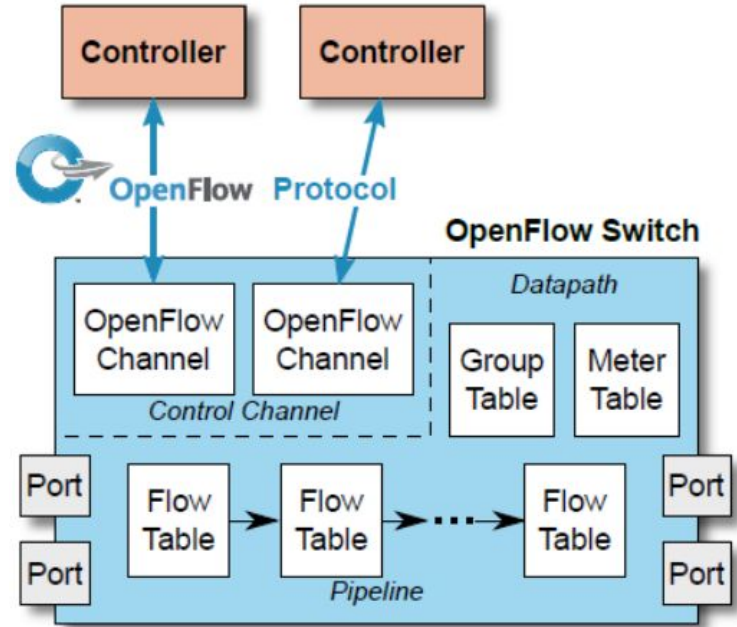


Datapath

- The OpenFlow specification defines three types of tables in the logical switch architecture
 - A **Flow Table** matches incoming packets to a particular flow and specifies the functions that have to be performed on the packets
 - There may be **multiple flow tables that operate in a pipeline fashion**
 - A flow table may direct a flow to a **Group Table**, which may trigger a variety of actions that affect one or more flows
 - A **Meter Table** can trigger a variety of performance-related actions on a flow
- An OpenFlow switch process packets by associating them to flows
- In general terms, **a flow is a sequence of packets** traversing a network **that share a set of header field values**

Device abstraction

- A number of ports (physical, logical, reserved)
- The pipeline, composed of one or more Flow Tables
 - – Packet lookups and forwarding
- Group table
 - – Advanced forwarding
- Meter table
 - – Statistics
- OpenFlow Channels
 - – Communication toward controller



OF Ports

- OpenFlow switches are connected through OpenFlow ports
 - Network interfaces to exchange packets with the rest of the network
- Types:
 - Physical Ports
 - Switch defined ports correspond to a hardware interface (e.g., map one-to-one to the Ethernet interfaces)
 - Logical Ports
 - Switch defined ports that do not correspond to a hardware switch interface (e.g. Tunnel-ID)
 - Reserved Ports
 - specify generic forwarding actions such as sending to the controller, flooding and forwarding using non-OpenFlow methods, such as normal switch processing

OF Ports

- Reserved Port Types (required)
 - **ALL**
 - Represents **all ports the switch can use for forwarding** a specific packets
 - Can be used only as output interface
 - **CONTROLLER**
 - Represents the **control channel with the OpenFlow controller**
 - Can be used as an ingress port or as an output port
 - **TABLE**
 - Represents the **start of the OpenFlow pipeline**
 - Submits the packet to the first flow table

OF Ports

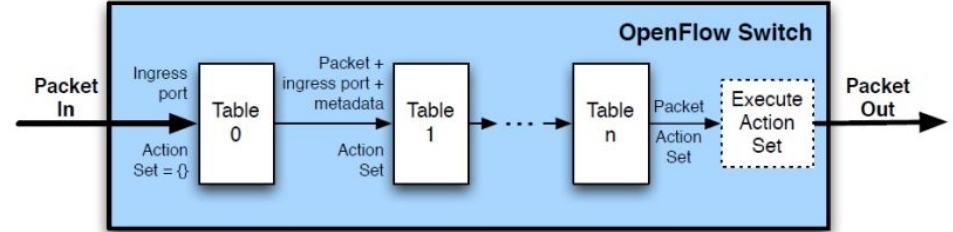
- Reserved Port Types (required)
 - **IN_PORT**
 - Represents the **packet ingress port**
 - Can be used only as an output port
 - **ANY**
 - Special value used in some OpenFlow commands when no port is specified
 - Can neither be used as an ingress port nor as an output port

OF Ports

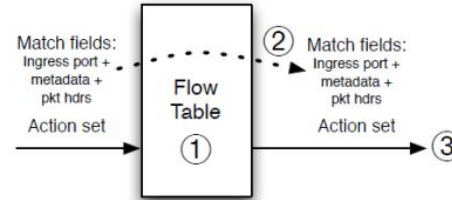
- Reserved Port Types (optional)
 - **LOCAL**
 - Represents the switch's **local networking stack** and its management stack
 - Can be used as an ingress port or as an output port
 - **NORMAL**
 - Represents the traditional **non-OpenFlow pipeline of the switch**
 - Can be used only as an output port and processes the packet using the normal pipeline
 - **FLOOD**
 - Represents **flooding** using the normal pipeline
 - Can be used only as an output port
 - Send the packet out on all ports except the incoming port and the ports that are in blocked state

Openflow Pipeline

- If matching found
 - Execute instructions
 - E.g., modify pck fields, update actions set
 - E.g., direct to another table, group or meter
- If match not found
 - Table miss default action
 - E.g., drop, send to controller
- After last table
 - Execute cumulated action set

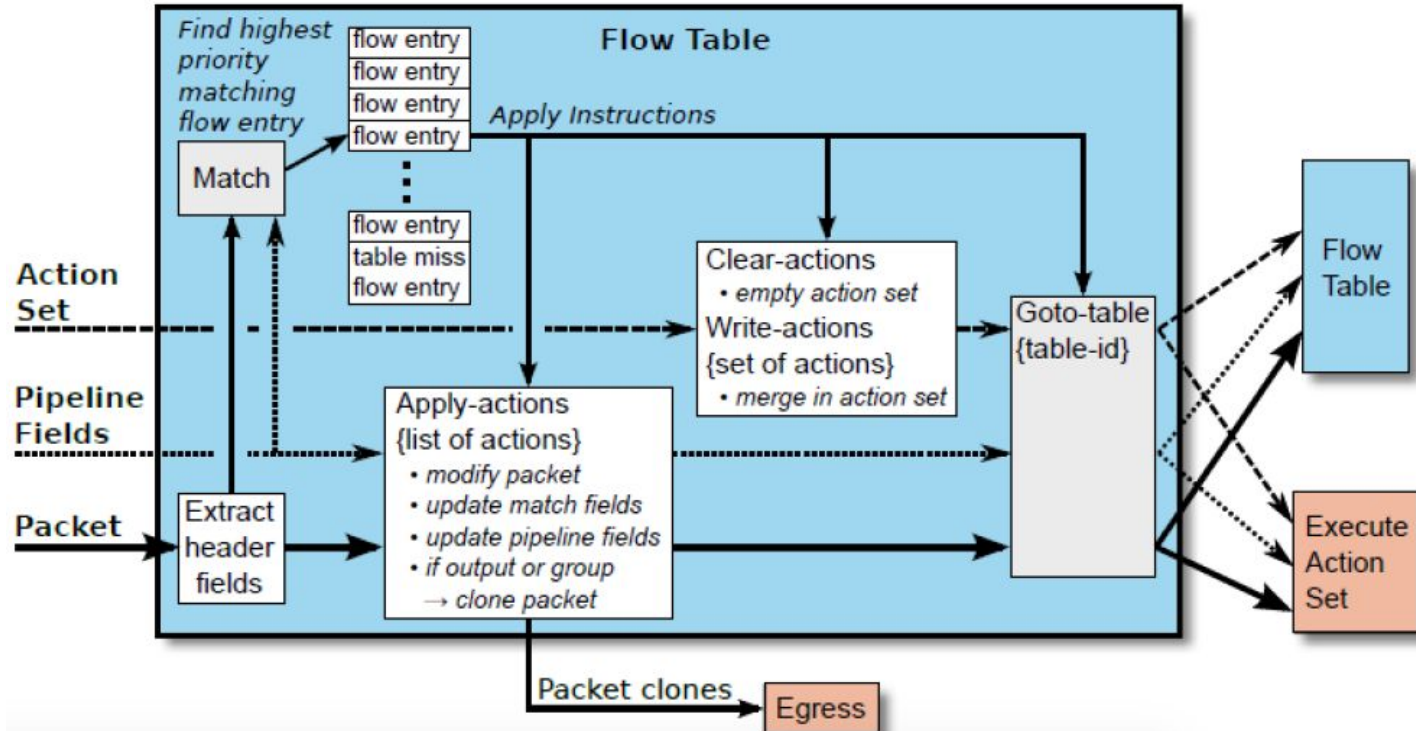


(a) Packets are matched against multiple tables in the pipeline

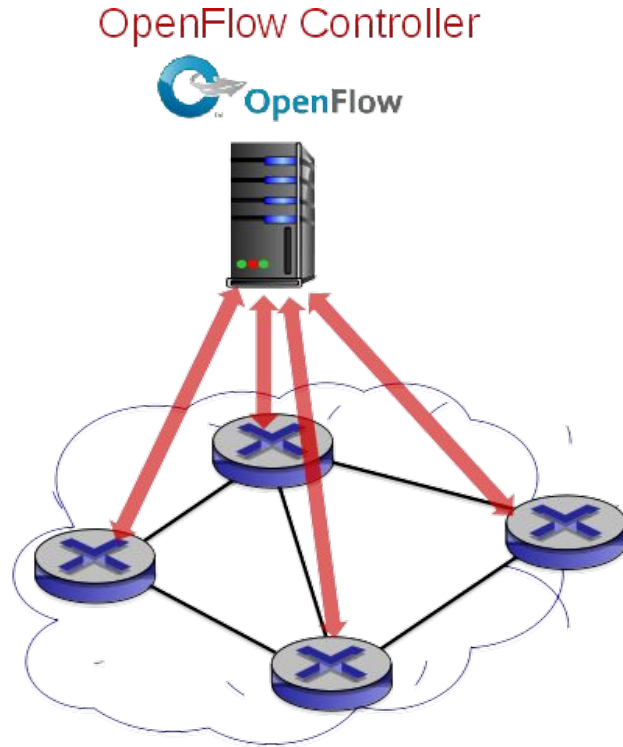


- ① Find highest-priority matching flow entry
- ② Apply instructions:
 - i. Modify packet & update match fields (apply actions instruction)
 - ii. Update action set (clear actions and/or write actions instructions)
 - iii. Update metadata
- ③ Send match data and action set to next table

Flow Matching



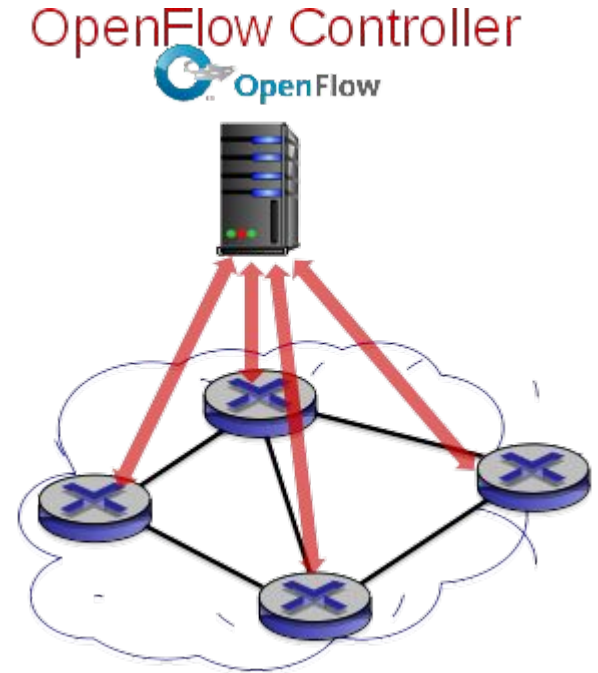
OF Messages



- operates between controller, switch
- TCP used to exchange messages
- optional encryption
- **three classes of OpenFlow messages:**
 - controller-to-switch
 - asynchronous (switch to controller)
 - symmetric (misc)

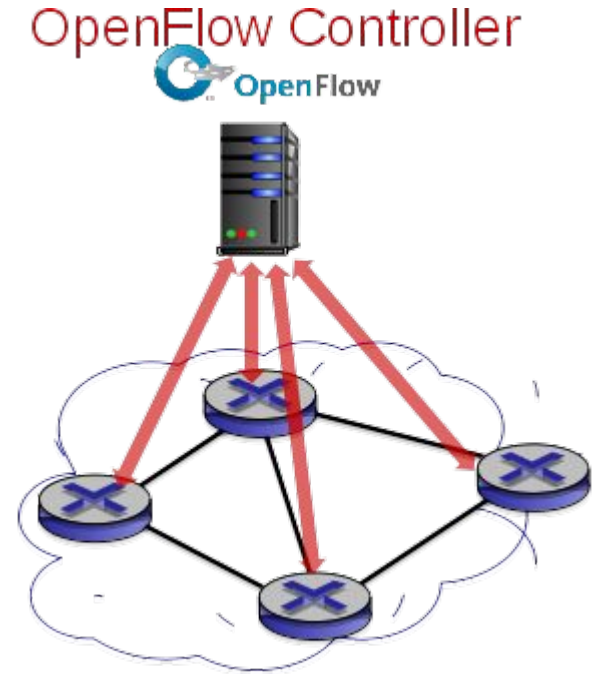
OF Messages

- **Key controller-to-switch messages**
 - **features:** controller queries switch features, switch replies
 - **configure:** controller queries/sets switch configuration parameters
 - **modify-state:** add, delete, modify flow entries in the OpenFlow tables
 - **packet-out:** controller can send this packet out of specific switch port



OF Messages

- **Key switch-to-controller messages**
 - **packet-in:** transfer packet (and its control) to controller. See packet-out message from controller
 - **flow-removed:** flow table entry deleted at switch
 - **port status:** inform controller of a change on a port



Flow Routing vs Aggregation

Flow-Based

- Every flow is individually set up by controller
- Exact-match flow entries
- Flow table contains one entry per flow
- Good for fine grain control, e.g. campus networks

Aggregated

- One flow entry covers large groups of flows
- Wildcard flow entries
- Flow table contains one entry per category of flows
- Good for large number of flows, e.g. backbone

Reactive vs Proactive

Reactive

- First packet of flow triggers controller to insert flow entries
- Efficient use of flow table
- Every flow incurs small additional flow setup time
- If control connection lost, switch has limited utility

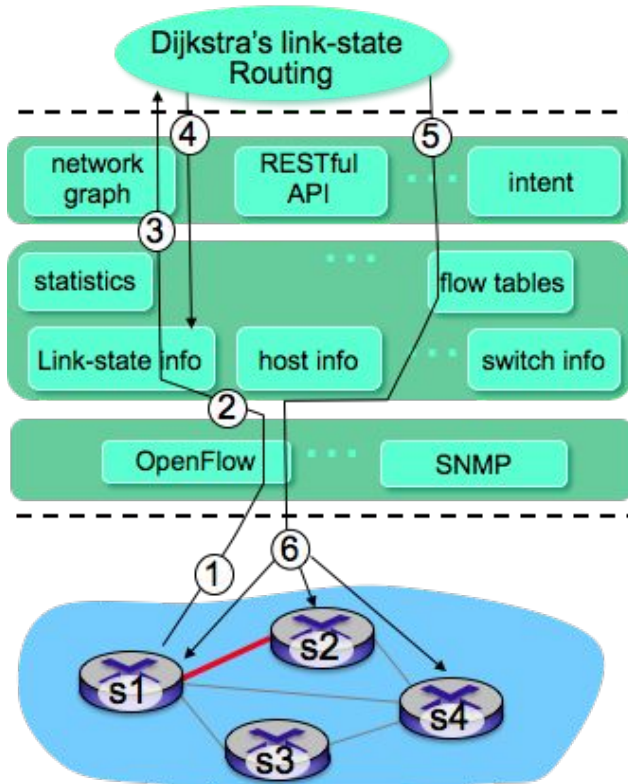
Proactive

- Controller pre-populates (a priori) flow table in switch
- Zero additional flow setup time
- Loss of control connection does not disrupt traffic
- Essentially requires aggregated (wildcard) rules

Outline

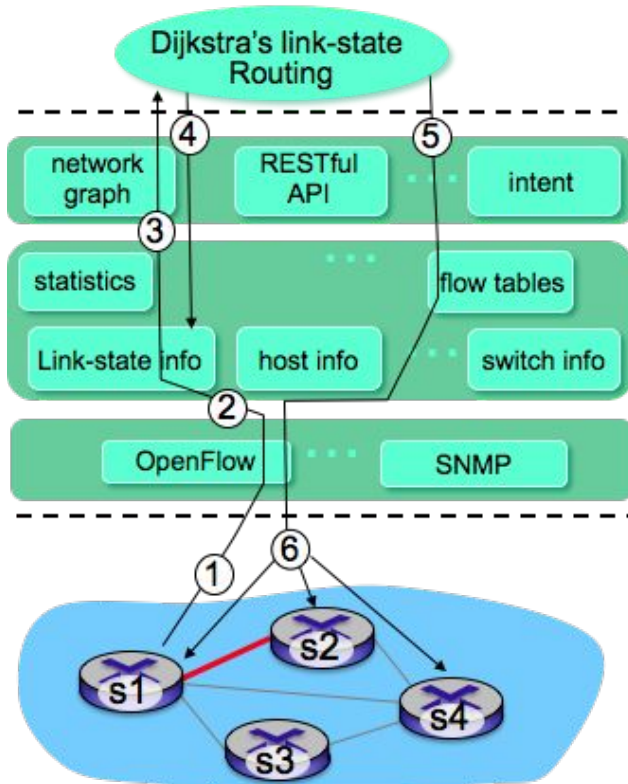
- Introduction
- Generalized Forwarding
- SDN Paradigm
- Openflow Protocol
- Data/Control Plane interactions in SDN

SDN control/data plane interaction example



1. S1, experiencing link failure using OpenFlow port status message to notify controller
2. SDN controller receives OpenFlow message, updates link status info
3. Dijkstra's routing algorithm application has previously registered to be called when ever link status changes. It is called.
4. Dijkstra's routing algorithm access network graph info, link state info in controller, computes new routes

SDN control/data plane interaction example



5. link state routing app interacts with flow-table-computation component in SDN controller, which computes new flow tables needed
6. Controller uses OpenFlow to install new tables in switches that need updating